

CS466 Lab 1 -- Hardware, Development Tools and Blinking the LED

Due by Midnight Friday 1-23-2026.

Must use provided lab format!!! (Format is soft, sections and content are important for full credit)

Note: This is an individual lab, you are free to collaborate but every student must perform the lab and hand in a lab report. It will be critical that each student is able to develop to the target platform.

I will be checking out a development board to each student. If you lose it, soak it, step on it, or let the smoke out of it during class you will be expected to replace it. They are very cheap and I will collect one per student at the end of the semester.

Overview:

We will be using a Raspberry Pi Pico board. It's essentially a minimal development board to showcase the RP2040 microcontroller. The ASIC has a processor and a bevy of integrated peripherals that the developer can combine and use to control an arbitrary device.

Benefits of the hardware:

- Very Affordable, [Currently around \\$5.00](#)
- Dual-core Arm Cortex M0+ processor, flexible clock running up to 133 MHz
- 264KB of SRAM, and 2MB of on-board Flash memory
- Castellated module allows soldering direct to carrier boards
- USB 1.1 with device and host support
- Low-power sleep and dormant modes
- Drag-and-drop programming using mass storage over USB
- 26 × multi-function GPIO pins
- 2 × SPI, 2 × I2C, 2 × UART, 3 × 12-bit ADC, 16 × controllable PWM channels
- Accurate clock and timer on-chip
- Temperature sensor
- Accelerated floating-point libraries on-chip
- 8 × Programmable I/O (PIO) state machines for custom peripheral support
- Open source Gnu tool chain, for Windows, Linux, OS-X
- Small and versatile.

Cons of the new hardware:

- No Snazzy IDE, command line tools and debugger.
- One Single-Color (green) LED
- Very little electrical protection on I/O pins (We need to be careful)
 - Can damage dev-board (we've roasted a couple)
 - Potentially could damage host computer USB hardware (we've not done this yet)

Resources:

- Raspberry Pi PICO DevBoard Documentation (<https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html>)
- Raspberry RP2040 Microcontroller Documentation (<https://www.raspberrypi.com/documentation/microcontrollers/rp2040.html#welcome-to-rp2040>)
- RP2040 Datasheet (<https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>)
- Raspberry Pi Pico SDK documentation (<https://raspberrypi.github.io/pico-sdk-doxygen/>)

Lab Preparation:

- Take some time to browse the above documentation. Don't install the development tools yet as I'd like everyone to have the same directory structure fir convince. We will be using a command line ARM GCC toolchain

- o Locate the board schematic. One of the lessons I learned in drafting this lab is that the button on the poco board is not directly available. There is an example in the pico devkit but it has to jump through hoops on failing to read the QSPI memory device on the board and assuming that is button press... We'll setup two GPIO pins to serve as two operator inputs.
- o Take a slightly longer look at the microcontroller datasheet. This is a large pdf (654 Pages).. While we will be looking into specific sections in detail for now, I want you to read sections 1 (16 pages), 2.1.9.1 and 2.1.9.2
 - o If you are not familiar with data sheets this can be a daunting document. Part of what we will be discussing in lab is how to not freak out when presented with all the data. The EE students have an early edge here having been exposed to data sheets before but the CS guys get it back later in the class.

Objective:

This lab is mostly to familiarize students with the Pico development board and development tools. Making use of some GPIO pins will require some basic understanding of the schematic and understanding some of the GPIO section of the microcontroller reference manual.

Lab Work

1. ☐ For LAB 1 I would like everyone to complete their work on a Linux system, their own or the lab computers
2. ☐ All of the lab material will be available on my git repo. Choose a directory to use to locate your cs466 project in (I recommend ~/src). From that directory run the command:

```
$ git clone https://github.com/milhead2/cs466\_s26
```

Using my standard this should create a directory ~/src/cs466_s25 with the first set of class files. Keep the directory tree in this format and support will be easier for your labs.

The README.md file on the class repo gives instruction on how to setup the directory and all the tools. Go read it now for an update on how to setup the Pico support and build requirements.

I only 'add' files to this repo so you are free to work in your copy of the repo. It would be 'more proper' to fork the repo. As I only add files, we should not see conflicts if you pull to get added files.

The repo README.md has data that should help you setup your dev environment

3. ☐ Familiarize yourself with the pico development board.
4. ☐ Check that the compiler is available on the system by running the commands that all should give a reasonable response. If your system does not have an **arm-none-eabi-gcc** application you will need to download and install it from <https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm/downloads>

```
$ arm-none-eabi-gcc -version
```

5. □ The Pico board does not have any default program loaded so when you plug it in no indication that the board is doing anything except for the occurrence of the directory above. In order to blink the LED we need to install the `blink` example program. So far I have had to reboot the pico with the button pressed to enter usb-drive mode.
 - a) This is cumbersome and will cause a lot of wear on the Micro-USB connector, I'll keep looking for another way to reload but that's it for now.
6. □ Get your development environment setup so that you can build the oversimplified Blinky program that will be provided in lab.
 - a) When you look at the class repo online the readme outlines the steps that I took to get everything compiled. Was a bit tricky and took me the lions share of this afternoon to get it worked out to your lab1 starting point.
7. □ Locate the schematic for the poco board on the board website. When compared to the lab1.c code is it clear what is occurring?
 - a) Question: Describe the expected operation of the lab1.c provided program
8. □ Verify that when you plug in the development board on Linux it is recognized by the OS. After you have the board plugged in and running, type a dmesg command to display the latest system stuff that occurred on the workstation;

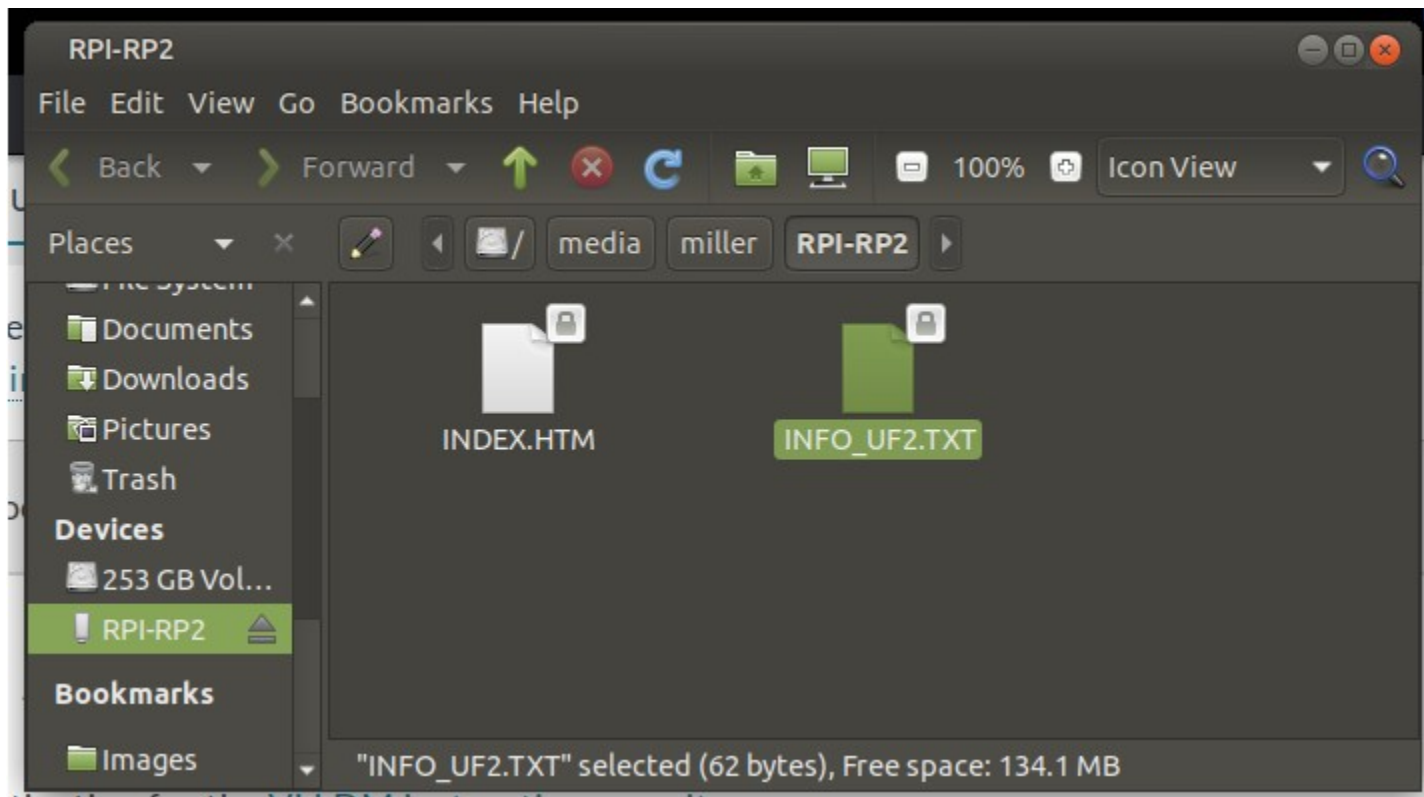
\$ dmesg

'dmesg' will display a gob of stuff but the last few lines are most interesting. You should see something like...

```
[102582.178254] usb 2-1.6: new full-speed USB device number 15 using ehci-pci
[102582.287864] usb 2-1.6: New USB device found, idVendor=2e8a, idProduct=0003, bcdDevice= 1.00
[102582.287869] usb 2-1.6: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[102582.287871] usb 2-1.6: Product: RP2 Boot
[102582.287874] usb 2-1.6: Manufacturer: Raspberry Pi
[102582.287876] usb 2-1.6: SerialNumber: E0C912952D54
[102582.288441] usb-storage 2-1.6:1.0: USB Mass Storage device detected
[102582.289099] scsi host10: usb-storage 2-1.6:1.0
[102583.299694] scsi 10:0:0:0: Direct-Access RPI RP2 2 PQ: 0 ANSI: 2
[102583.300536] sd 10:0:0:0: Attached scsi generic sg5 type 0
[102583.301990] sd 10:0:0:0: [sde] 262144 512-byte logical blocks: (134 MB/128 MiB)
[102583.303177] sd 10:0:0:0: [sde] Write Protect is off
[102583.303182] sd 10:0:0:0: [sde] Mode Sense: 03 00 00 00
[102583.304309] sd 10:0:0:0: [sde] No Caching mode page found
[102583.304317] sd 10:0:0:0: [sde] Assuming drive cache: write through
[102583.351867] sde: sde1
[102583.382682] sd 10:0:0:0: [sde] Attached SCSI removable disk
```

This shows that Linux has seen the device and understands what it is.

In the default configuration the board will load the last program you downloaded. If you don't see the board appear as a USB drive and display a file browser like;



Then unplug the board and hold down the BOOTSEL button while you plug it in.

- Raspberry claims that you cannot brick this board and holding the BOOTSEL button will ‘always’ return the board to default waiting-for-image program... We’ll see if that’s true.

9. □ In order to proceed with the remaining lab steps, we will need three additional GPIO’s in play. Setup pico board pins 21 and 22 as inputs and 24 as an output.

Be careful here (I wasted an hour for something I should have known) that the pico board pin numbers are mapped to different GPIO numbers



To setup board pin 24 as an output, you will be dealing with GP18 in the SDK. Similarly pico-board 21 and 22 are GPIO’s 16 and 17 respectively.

10. □ In the code create a `led_control(bool isOn)` function and call it in place of the supplied `gpio_put()` that is used in the main loop. In the `led_control()` function I want you to not only light and extinguish the

LED but set the state of the GP18 output high and low at the same time as you switch the LED. Measure the frequency of the blinking LED by using a scope on the GP18 Output

Display the waveform of the blinking LED on a scope.

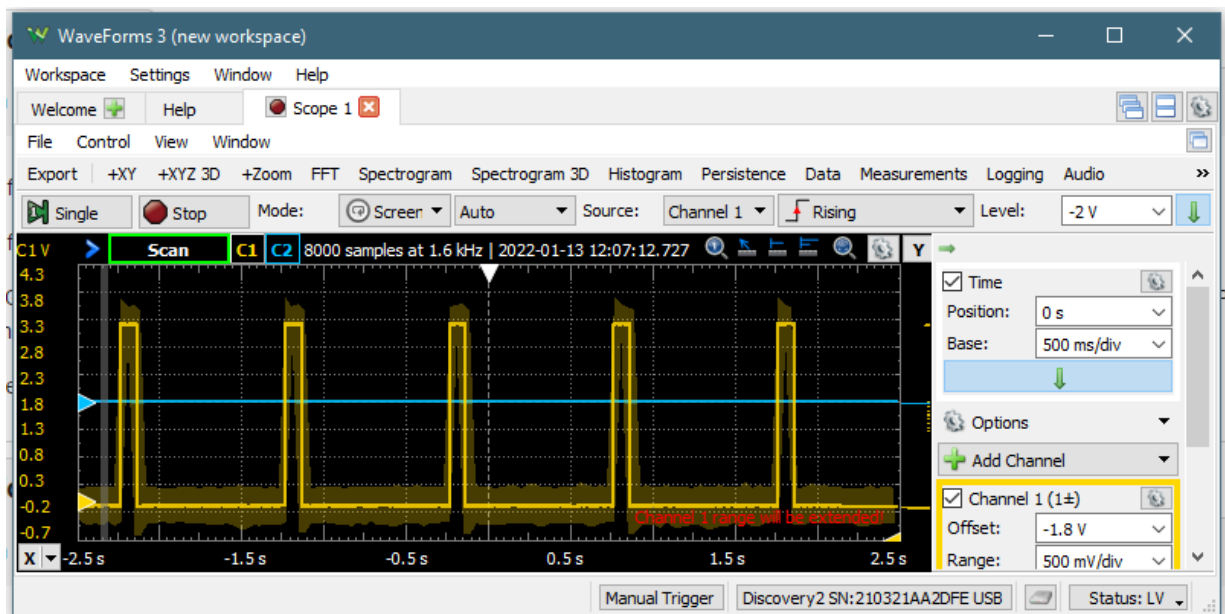
a) Lab Writeup: Does it match the expectation that you set earlier? Include the scope data (Phone Images work)

11. □ Add the ability to read from pins GP16 and GP17. I want you to set the pins up to be pulled up internally (check SDK). In this way they will read high if nothing is connected to the pin and will read low if the pin is shorted to ground. We will use these as a user-input switches SW1 and SW2. When pressed it will short the GP16 and GP17 pins to ground respectively.

Devise a way using only the LED to test if SW1, SW2, or both are shorted to ground (pressed). You have only the LED flashing to indicate GPIO inputs at this time.

- a) NOTE: Don't proceed until you have this test passing or the remaining sections of the lab will get very hard.
- b) Lab Writeup: Give a detailed description of your test strategy here, imagine you are documenting this to be implemented by your peers.
- c) Lab Writeup: Describe the results and any conclusions from your test.

On one of my conditions, I set up the LED to be on for 100ms then off for 900ms. Monitoring GP18 with the scope gave me;



12. □ Modify the code so that:

- a) If no buttons are pressed, the green LED continuously blinks at 1.0Hz
- b) Pressing SW1 will cause the led to flash 20 times at 15Hz
- c) Pressing SW2 will cause the led to flash 10 times at 13Hz
- d) If both buttons are pressed the LED should flash continuously at 5 Hz.
- e) For single buttons, use only the button press as a trigger, the led should blink their required number of times no matter how long the button is pressed.

13. □ Verify the frequencies (or as close as you can get) with the scope.

a) Lab Writeup: Show the scope data and describe it in your writeup.

14. Write up your lab using the lab format provided on Blackboard. Include your program as a fixed spaced (I recommend Lucida Console) addendum to your lab. I will cut points for proportionally spaced code pasted in the end of the lab.

Submit your lab as a 'SINGLE' PDF file on Blackboard. Name the file
CS466L01_Name(s)_Report.pdf

.... Not a .zip File

.... Not a set of files

.... Not a set of .jpg files from your phone. (Yes, someone actually did this.... (sigh))

(I'm not too much a jerk. Using a single PDF makes grading, feedback and organization of reports easier for everyone)